

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
з дисципліни «Інженерія систем IoT»
Протокол MQTT

Виконав:
студент групи ІА-34
Швець Роман Вадимович

Перевірив:
асистент кафедри ІСТ
Головатенко І. А.

Мета: Метою даної лабораторної роботи є підключення нічника IoT до Інтернету для забезпечення можливості дистанційного керування. В подальшій частині роботи пристрій IoT надішле телеметричне повідомлення через протокол MQTT до загальнодоступного брокера MQTT із даними про рівень освітленості, де вони будуть оброблені спеціально написаним серверним кодом. Цей код перевірить рівень освітленості та надішле команду назад до пристрою з вказівкою увімкнути або вимкнути світлодіод.

Хід роботи

Ми повинні обрати свій варіант завдання згідно з номером у списку потоку.

109	Швець Роман
-----	-------------

Варіант 109

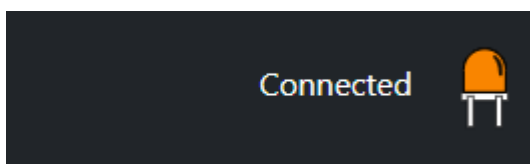
1. Створіть датчик світла на 109 контакті (Pin).
2. Створіть LED на 110 контакті (Pin).
3. Встановіть перевірку рівня освітленості на 1090.
4. Використовуйте `f88b0c4e-a8c3-4525-a02c-d31bafef43a7b` як ідентифікатор пристрою.
5. Продемонструйте включення та вимкнення LED залежно від рівня освітленості.

Запускаємо Counterfit так, як ми робили це в попередніх лабораторних роботах.

В терміналі запускаємо програму CounterFit, прописуючи:

```
counterfit
```

Отримуємо сповіщення про те, що програма працює на порті 5000. Запускаємо файл `app.py`, щоб підключитись до інтерфейсу. Після запуску файлу в інтерфейсі CounterFit побачимо успішне підключення:



Маємо такий датчик світла:

Light	Pin 109
Units:	NoUnits
Value range:	0 to 1,023
Value:	107
Random:	☒
Min	Max
0	218
Set	Delete

Також маємо світлодіод на 110 контакті:

LED	Pin 110
Color:	<input type="color" value="#800080"/>
	
Set	Delete

Запустимо через консоль файл `app.py`, який після змін виглядає таким чином:

```
from counterfit_connection import CounterFitConnection
CounterFitConnection.init('127.0.0.1', 5000)

import time
from counterfit_shims_grove.grove_light_sensor_v1_2 import GroveLightSensor
from counterfit_shims_grove.grove_led import GroveLed
import paho.mqtt.client as mqtt

light_sensor = GroveLightSensor(109)
```

```

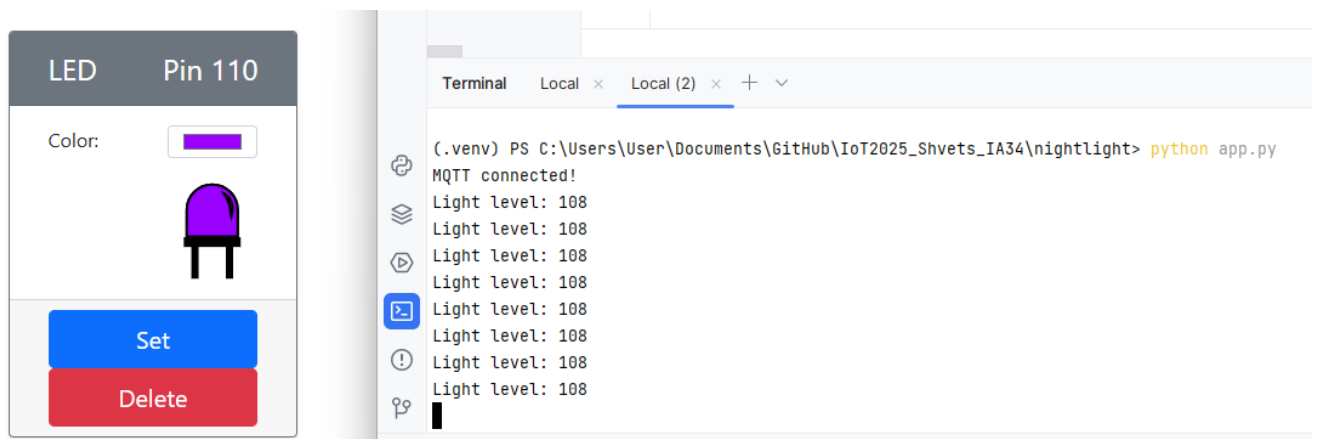
led = GroveLed(110)
id = 'f88b0c4e-a8c3-4525-a02c-d31baf4e43a7b'
client_name = id + 'nightlight_client'
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
print("MQTT connected!")
while True:
    light = light_sensor.light
    print('Light level:', light)
    if light < 109:
        led.on()
    else:
        led.off()

    time.sleep(3)

```

Ми додали MQTT-підключення, тепер програма не лише керує світлодіодом, а й може обмінюватися даними через протокол MQTT.

Запускаємо програму на виконання:



Бачимо підтвердження MQTT-підключення та те, що при рівні світла меншому за 109 наш світлодіод загоряється.

Тепер потрібно реалізувати надсилання телеметрії з IoT-пристрою. Для цього додамо такі фрагменти коду

```

import json

client_telemetry_topic = id + '/telemetry'

```

Також замінимо наш цикл While на такий:

```

while True:
    light = light_sensor.light
    telemetry = json.dumps({'light': light})
    print("Sending telemetry:", telemetry)
    mqtt_client.publish(client_telemetry_topic, telemetry)
    time.sleep(5)

```

Отримаємо такий вивід:

```
(.venv) PS C:\Users\User\Documents\GitHub\IoT2025_Shvets_IA34\nightlight> python app.py
MQTT connected!
Sending telemetry: {"light": 108}
Sending telemetry: {"light": 108}
```

Далі створюємо папку nightlight-server та віртуальне середовище всередині неї, також створюємо файл app.py з таким вмістом:

```
import json
import time
import paho.mqtt.client as mqtt
id = 'f88b0c4e-a8c3-4525-a02c-d31baf43a7b'
client_telemetry_topic = id + '/telemetry'
client_name = id + 'nightlight_server'
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
def handle_telemetry(client, userdata, message):
    payload = json.loads(message.payload.decode())
    print("Message received:", payload)

mqtt_client.subscribe(client_telemetry_topic)
mqtt_client.on_message = handle_telemetry
while True:
    time.sleep(2)
```

Отримуємо такий вивід:

```
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
Message received: {'light': 108}
```

Додаємо код для визначення теми команди та код, який задає нашу команду до нашого файлу. Фінальний вигляд app.py для «серверної частини»:

```
import json
import time
import paho.mqtt.client as mqtt
id = 'f88b0c4e-a8c3-4525-a02c-d31baf43a7b'
client_telemetry_topic = id + '/telemetry'
server_command_topic = id + '/commands'
client_name = id + 'nightlight_server'
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
def handle_telemetry(client, userdata, message):
    payload = json.loads(message.payload.decode())
```

```

print("Message received:", payload)
command = {'led_on': payload['light'] < 109}
print("Sending message:", command)
client.publish(server_command_topic, json.dumps(command))

mqtt_client.subscribe(client_telemetry_topic)
mqtt_client.on_message = handle_telemetry
while True:
    time.sleep(2)

```

Додаємо відповідний код до нашого IoT пристрою. Фінальна версія:

```

from counterfit_connection import CounterFitConnection
CounterFitConnection.init('127.0.0.1', 5000)

import json
import time
from counterfit_shims_grove.grove_light_sensor_v1_2 import GroveLightSensor
from counterfit_shims_grove.grove_led import GroveLed
import paho.mqtt.client as mqtt

light_sensor = GroveLightSensor(109)

led = GroveLed(110)
id = 'f88b0c4e-a8c3-4525-a02c-d31bafe43a7b'
client_name = id + 'nightlight_client'
client_telemetry_topic = id + '/telemetry'
server_command_topic = id + '/commands'
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
def handle_command(client, userdata, message):
    payload = json.loads(message.payload.decode())
    print("Message received:", payload)
    if payload['led_on']:
        led.on()
    else:
        led.off()
mqtt_client.subscribe(server_command_topic)
mqtt_client.on_message = handle_command
print("MQTT connected!")
while True:
    light = light_sensor.light
    telemetry = json.dumps({'light': light})
    print("Sending telemetry:", telemetry)
    mqtt_client.publish(client_telemetry_topic, telemetry)
    time.sleep(5)

```

Отримаємо такий вивід:

```

Sending telemetry: {"light": 24}
Message received: {'led_on': True}
Sending telemetry: {"light": 176}
Message received: {'led_on': False}

```

Висновок: У ході лабораторної роботи ми успішно налаштували наш IoT-пристрій для відповіді на команди від MQTT-брокера. Ми переконалися, що при зміні рівня освітлення поведінка світлодіода змінюється відповідним чином.

https://github.com/rshvtss/IoT2025_Shvets_IA34